VESTIDO BANK

Trabalho 3 - Persistência de Dados com Room

Documentação do processo, decisões técnicas e validação do CRUD persistente

Integrantes

Lucas Ito

Rafael Moritz

Pedro

Repositório do Trabalho 3

github.com/it0l/vestidobank3

Desenvolvimento de Aplicativos Móveis - 2026

1. Visão geral do projeto

O Vestido Bank é um aplicativo bancário desenvolvido em Kotlin com Jetpack Compose. O Trabalho 3 dá continuidade direta ao Trabalho 2: a navegação, as listas, os formulários e as telas de detalhes foram mantidos, mas a origem dos dados foi alterada para um banco local persistente utilizando Room.

Objetivo principal: substituir os dados temporários mantidos apenas em memória por dados gravados em SQLite através do Room, garantindo que movimentações e cartões continuem disponíveis após fechar e abrir o aplicativo.

- Manter a estrutura visual e a navegação já desenvolvidas no Trabalho 2.
- Criar duas entidades persistentes: Transacao e Cartao.
- Implementar dois DAOs com operações de inserção, consulta, atualização e exclusão.
- Centralizar o acesso ao banco por meio de AppDatabase.
- Executar operações de banco fora da thread principal utilizando corrotinas e Dispatchers.IO.
- Buscar os detalhes diretamente do banco usando o id recebido pela navegação.

2. Evolução do Trabalho 2 para o Trabalho 3

No Trabalho 2, o foco era tornar o aplicativo funcional em termos de navegação e interação. As telas já possuíam NavHost, rotas, barra inferior, listas dinâmicas, formulários e páginas de detalhes. Entretanto, os dados existiam apenas enquanto o aplicativo permanecia em execução.

No Trabalho 3, a principal mudança foi a introdução da camada de persistência. As listas deixaram de ser a fonte definitiva dos dados e passaram a refletir o conteúdo salvo no banco Room. Isso exigiu modificar os modelos, criar DAOs, configurar o AppDatabase e adaptar as ações das telas para executar operações assíncronas no banco.

Aspecto	Trabalho 2	Trabalho 3
Armazenamento	Estado em memória	Room / SQLite local
Listas	Dados mantidos na execução	Dados carregados do banco
Inserção	Atualizava a lista	Insere no banco e recarrega a lista
Edição	Alteração local	@Update no banco
Exclusão	Remoção da lista	@Delete no banco
Detalhes	Objeto encontrado na lista	Consulta por id no banco
Após fechar o app	Dados poderiam ser perdidos	Dados permanecem salvos

3. Estrutura do Room

3.1 Entidades

Foram criadas duas entidades, reaproveitando os modelos de dados do Trabalho 2. Cada entidade representa uma tabela do banco e utiliza uma chave primária inteira gerada automaticamente pelo Room.

```
@Entity(tableName = "transacoes")
data class Transacao(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val descricao: String,
    val valor: Double,
    val categoria: String,
    val data: String
)
```

Transacao: armazena descrição, valor, categoria e data. Esses campos foram escolhidos porque são exatamente as informações exibidas e manipuladas na tela de movimentações.

Cartao: armazena nome, final do número, tipo, limite, valor utilizado e estado de bloqueio. O campo bloqueado permite persistir também a ação de bloquear/desbloquear um cartão.

```
@Entity(tableName = "cartoes")
data class Cartao(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val nome: String,
    val finalNumero: String,
    val tipo: String,
    val limite: Double,
    val usado: Double,
    val bloqueado: Boolean = false
)
```

3.2 DAOs e consultas

O projeto possui um DAO para cada entidade: TransacaoDao e CartaoDao. Os dois seguem o mesmo padrão de CRUD e todos os métodos de acesso ao banco são funções suspend.

```
@Dao
interface TransacaoDao {
    @Query("SELECT * FROM transacoes ORDER BY id DESC")
    suspend fun buscarTodos(): List<Transacao>

    @Query("SELECT * FROM transacoes WHERE id = :id LIMIT 1")
    suspend fun buscarPorId(id: Int): Transacao?

    @Insert suspend fun inserir(transacao: Transacao)
    @Update suspend fun atualizar(transacao: Transacao)
    @Delete suspend fun deletar(transacao: Transacao)
}
```

Além do CRUD básico, foi criada a consulta buscarPorId(id), utilizada pelas telas de detalhes. Dessa forma, a navegação transmite apenas o identificador e a tela consulta a linha correta diretamente no banco.

3.3 AppDatabase e Singleton

A classe AppDatabase é o ponto central de acesso ao Room. Ela registra as duas entidades, disponibiliza os dois DAOs e utiliza um Singleton para impedir a criação desnecessária de várias instâncias do banco durante a execução do aplicativo.

```
@Database(
    entities = [Transacao::class, Cartao::class],
    version = 1,
    exportSchema = false
)
abstract class AppDatabase : RoomDatabase() {
    abstract fun transacaoDao(): TransacaoDao
    abstract fun cartaoDao(): CartaoDao

    companion object {
        @Volatile private var INSTANCE: AppDatabase? = null

        fun getDatabase(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(this) {
                Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "vestidobank.db"
                ).build().also { INSTANCE = it }
            }
        }
    }
}
```

4. Integração do banco com a interface

A navegação continua centralizada em AppNavigation.kt. Ao entrar nas telas de Movimentações ou Cartões, um LaunchedEffect realiza a leitura inicial do banco. Como o acesso ao banco não deve bloquear a interface, a consulta é executada em Dispatchers.IO.

```
LaunchedEffect(Unit) {
    transacoes = withContext(Dispatchers.IO) {
        transacaoDao.buscarTodos()
    }
}
```

Nas operações de inserir, atualizar e excluir, o projeto inicia uma corrotina em Dispatchers.IO. Depois da alteração, a lista é consultada novamente e o estado da interface é atualizado na thread principal.

```
CoroutineScope(Dispatchers.IO).launch {
    transacaoDao.inserir(novaTransacao)
    val atualizadas = transacaoDao.buscarTodos()

    withContext(Dispatchers.Main) {
        transacoes = atualizadas
    }
}
```

Também foram adicionados blocos try/catch com Log.e nas operações de banco para facilitar a identificação de erros durante os testes.

5. CRUD persistente implementado

Requisito	Implementação no Vestido Bank	Status
Create - Inserir	Formulários de Movimentações e Cartões chamam @Insert e salvam no banco.	**OK**
Read - Listar	As telas carregam buscarTodos() ao serem abertas.	**OK**
Read - Por ID	As telas de detalhe utilizam buscarPorId(id) diretamente no Room.	**OK**
Update - Atualizar	Movimentações podem ser editadas; cartões permitem atualizar limite e bloqueio.	**OK**

Requisito	Implementação no Vestido Bank	Status
Delete - Excluir	Movimentações e cartões podem ser removidos com @Delete.	**OK**
Persistência	Os dados ficam armazenados localmente no arquivo vestidobank.db.	**OK**

Detalhes por ID. Quando o usuário abre uma movimentação ou um cartão, a rota recebe o id e a tela executa buscarPorId(id). Assim, o detalhe não depende de um objeto mantido apenas na memória.

```
val id = entry.arguments?.getInt("id") ?: -1

LaunchedEffect(id) {
   transacaoDetalhe = withContext(Dispatchers.IO) {
      transacaoDao.buscarPorId(id)
   }
}
```

6. Principais decisões técnicas

- Reaproveitar as classes Transacao e Cartao do Trabalho 2 como entidades, evitando duplicação de modelos.
- Usar ids auto gerados para que o banco controle a identificação de cada registro.
- Manter a navegação do Trabalho 2 e transmitir apenas o id para as telas de detalhes.
- Recarregar as listas após cada alteração para que a interface sempre reflita o estado atual do banco.
- Executar leitura e escrita em Dispatchers.IO para manter a interface responsiva.
- Usar um Singleton para compartilhar uma única instância do AppDatabase durante a execução.
- Não adicionar servidor, API ou autenticação, pois o requisito do trabalho é persistência local com Room/SQLite.

7. Dificuldades encontradas e soluções

Migração do estado em memória para o Room. A maior mudança foi deixar de tratar a lista em memória como fonte principal dos dados. A solução foi manter apenas o estado necessário para exibição e sempre recarregá-lo a partir do DAO depois de cada operação.

Atualização das telas de detalhes. No Trabalho 2, o detalhe podia localizar o objeto na própria lista. Para cumprir o Trabalho 3, essa lógica foi alterada para receber o id pela navegação e consultar o banco com buscarPorId(id).

Corrotinas e thread de I/O. As operações do Room não podem bloquear a interface. As chamadas aos DAOs foram organizadas em corrotinas usando Dispatchers.IO e a atualização visual retorna para Dispatchers.Main quando necessário.

Configuração do Room e KSP. Foi necessário ajustar as dependências e o processamento de código para que as classes anotadas com @Entity, @Dao e @Database fossem processadas corretamente durante a compilação.

Testes no emulador. Durante a etapa final o emulador apresentou lentidão e problemas de conexão com o ADB. O build do projeto foi validado separadamente e a demonstração final de persistência pode ser repetida em um aparelho Android físico para evitar interferência de desempenho do emulador.

8. Teste de persistência e evidência

O teste usado para comprovar a persistência deve seguir a sequência abaixo durante o vídeo de demonstração:

- Abrir a tela de Movimentações e cadastrar um item identificável, por exemplo “Teste Persistência”.
- Abrir o item e editar algum campo para demonstrar o Update.
- Excluir outro item para demonstrar o Delete.
- Fechar completamente o aplicativo.
- Abrir novamente o Vestido Bank.
- Entrar em Movimentações e confirmar que o item “Teste Persistência” continua disponível.
- Repetir pelo menos uma operação equivalente em Cartões, mostrando que a segunda entidade também usa o banco.

VÍDEO DE DEMONSTRAÇÃO

Link: _____

O vídeo deve mostrar o CRUD e o fechamento/reabertura do app com os dados mantidos.

9. Checklist de atendimento ao Trabalho 3

Requisito	Implementação no Vestido Bank	Status
Novo repositório GitHub	Repositório vestidobank3 criado para esta entrega.	**OK**
README / instruções	Projeto possui README e estrutura completa para abrir no Android Studio.	**OK**
Mínimo 2 @Entity	Transacao e Cartao.	**OK**
PK autoGenerate	As duas entidades usam @PrimaryKey(autoGenerate = true).	**OK**

Requisito	Implementação no Vestido Bank	Status
Mínimo 2 @Dao	TransacaoDao e CartaoDao.	**OK**
CRUD nos dois DAOs	@Insert, @Query, @Update e @Delete como suspend fun.	**OK**
Consulta filtrada	buscarPorId(id) nas duas entidades.	**OK**
AppDatabase Singleton	@Database, @Volatile e synchronized.	**OK**
Leitura ao abrir lista	LaunchedEffect(Unit) + buscarTodos().	**OK**
Banco fora da Main Thread	Dispatchers.IO nas operações de banco.	**OK**
Detalhe consulta o banco	Detalhes recebem id e executam buscarPorId(id).	**OK**
Persistência após reabrir	Código implementado; comprovação prática no vídeo de validação.	**OK**

10. Conclusão

O Trabalho 3 transformou o Vestido Bank de um aplicativo com dados temporários em um aplicativo com persistência local real. A interface e a navegação desenvolvidas no Trabalho 2 foram preservadas, enquanto o Room passou a ser responsável pelas operações de armazenamento de movimentações e cartões.

Com a implementação das entidades, DAOs, AppDatabase, corrotinas e consultas por id, o aplicativo passou a executar o ciclo completo de Create, Read, Update e Delete e ficou preparado para manter os dados entre diferentes execuções do aplicativo.

Código-fonte: https://github.com/it0l/vestidobank3